Miniproject: Simple Banking

Aim: To develop and deploy a decentralized **Simple Banking** application on the Ethereum Sepolia Testnet where:

- Every new account registers and receives **100 BANK tokens** from a total supply of 1,000,000 tokens.
- Users can deposit ETH, withdraw their deposited ETH, and transfer BANK tokens.
- ETH and BANK token balances can be retrieved using dedicated smart-contract view functions.
- A responsive Web3 frontend provides wallet connection, registration, transactions, balance checking, account listing, and liquidity checking.

Technology Stack Used

Solidity	Writes the smart contract for registration, ETH deposits and withdrawals, BANK transfers, balances, mappings, validation, and events.
HTML	Builds the page structure, including buttons, forms, balance cards, account status, and registered-account listing.
CSS	Creates the responsive black-and-white interface, typography, spacing, loading states, and validation feedback.
JavaScript	Handles user actions, form validation, loading states, wallet changes, and frontend updates.
ethers.js	Connects the browser to MetaMask and calls the deployed contract through its address and ABI.
Remix IDE	Compiles and deploys SimpleBank.sol and provides tools for contract testing.
MetaMask	Provides accounts, connects the DApp to Sepolia, signs transactions, and pays testnet gas fees.
Ethereum Sepolia	Provides a public test network where the project can be demonstrated using test ETH with no real monetary value.

Solidity Functions Used

registernewaccount()	Registers a new wallet and awards 100 BANK tokens.
deposit()	Accepts ETH and adds it to the caller's deposited balance.
withdraw(amount)	Returns the requested deposited ETH to the registered caller.
transfer(to, amount)	Moves BANK tokens between two registered accounts.
getEthBalance(user)	Returns the ETH deposited by a specified account.
getTokenBalance(user)	Returns the BANK token balance of a specified account.
getAccounts()	Returns the list of all registered wallet addresses.
contractBalance()	Returns the total ETH currently held by the contract.
registered(address)	Checks whether a wallet address has already registered.

Implementation Steps

1. Wallet & Testnet Setup

- Installed MetaMask and used it as the browser Web3 wallet provider.
- Connected MetaMask to the Ethereum Sepolia Testnet.
- Obtained free Sepolia ETH from the Google Cloud Web3 Faucet for gas fees.

2. Smart Contract Development & Compilation

- Created SimpleBank.sol using Solidity 0.8.20 with separate ETH and BANK token balances.
- Compiled the contract successfully in Remix IDE.

3. On-Chain Contract Deployment

- Selected Browser Extension / MetaMask in Remix and verified the Sepolia network.
- Deployed the contract, approved the transaction in MetaMask, and copied its contract address.

4. Frontend Development & Web3 Integration

- Created a responsive index.html interface for wallet connection and all banking operations.
- Used ethers.js, the deployed contract address, and the ABI to connect buttons to contract functions.
- Added automatic Sepolia switching, input validation, loading indicators, and registered-account status.

5. Deployment & Multi-User Testing

- Served the frontend locally with VS Code Live Server at http://127.0.0.1:5500.
- Connected **Account 1:** Registered the account, verified receipt of 100 tokens, deposited 0.01 ETH, and executed a partial withdrawal.
- Added **Account 2** in MetaMask: Funded it with gas ETH, registered it, and performed peer-to-peer token transfers between both accounts to verify state updates on the blockchain.

1.Contract Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SimpleBank {
    // Total supply of 1,000,000 tokens and 100 token bonus per account
    uint256 public constant INITIAL_BALANCE = 100;
    uint256 public remainingSupply = 1000000;

    mapping(address => uint256) private ethBalances;
    mapping(address => uint256) private tokenBalances;
    mapping(address => bool) public registered;
    address[] public accounts;

    event AccountRegistered(address indexed user, uint256 tokenBonus);
    event Deposited(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);
    event Transferred(address indexed from, address indexed to, uint256 amount);

    modifier onlyRegistered() {
        require(registered[msg.sender], "Account not registered");
        _;
    }

    // 1. registernewaccount(): Registers user & allocates 100 tokens from 1,000,000 supply
    function registernewaccount() external {
        require(!registered[msg.sender], "Account already registered");
        require(remainingSupply >= INITIAL_BALANCE, "Supply exhausted");
```

```
    registered[msg.sender] = true;
    tokenBalances[msg.sender] = INITIAL_BALANCE;
    remainingSupply -= INITIAL_BALANCE;
    accounts.push(msg.sender);

    emit AccountRegistered(msg.sender, INITIAL_BALANCE);
}

// 2. deposit() -> payable: Accepts ETH and credits user balance
function deposit() external payable onlyRegistered {
    require(msg.value > 0, "Deposit must be greater than 0");
    ethBalances[msg.sender] += msg.value;
    emit Deposited(msg.sender, msg.value);
}

// 3. withdraw(uint amount) -> withdraws deposited ETH
function withdraw(uint256 amount) external onlyRegistered {
    require(amount > 0, "Amount must be greater than 0");
    require(ethBalances[msg.sender] >= amount, "Insufficient ETH balance");

    ethBalances[msg.sender] -= amount;
    (bool success, ) = payable(msg.sender).call{value: amount}("");
    require(success, "ETH transfer failed");

    emit Withdrawn(msg.sender, amount);
}

// 4. transfer(address to, uint amount) -> transfers tokens between users
function transfer(address to, uint256 amount) external onlyRegistered {
    require(to != address(0), "Invalid recipient");
    require(registered[to], "Recipient not registered");
    require(amount > 0, "Amount must be greater than 0");
```

```
        require(tokenBalances[msg.sender] >= amount, "Insufficient token balance");

        tokenBalances[msg.sender] -= amount;
        tokenBalances[to] += amount;

        emit Transferred(msg.sender, to, amount);
    }

    // 5. balanceOf(address user) -> returns token balance
    function balanceOf(address user) external view returns (uint256) {
        return tokenBalances[user];
    }

    // Helper view functions
    function getEthBalance(address user) external view returns (uint256) {
        return ethBalances[user];
    }

    function getAccounts() external view returns (address[] memory) {
        return accounts;
    }

    function contractBalance() external view returns (uint256) {
        return address(this).balance;
    }
}
```

Frontend Code:

```
  <!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <meta name="theme-color" content="#2563eb">
   <title>Simple Banking App</title>
   <link rel="preconnect" href="https://fonts.googleapis.com">
   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
   <link
href="https://fonts.googleapis.com/css2?family=Geist:wght@400;500;600&family=Geist+Mono:wght@400
;500&display=swap" rel="stylesheet" referrerpolicy="no-referrer">
   <script src="https://cdn.jsdelivr.net/npm/ethers@6.13.4/dist/ethers.umd.min.js"></script>
   <style>
      :root {
         --primary: #2563eb;
         --primary-dark: #1d4ed8;
         --primary-soft: #eff6ff;
         --ink: #101828;
         --muted: #667085;
         --line: #e7eaf0;
         --background: #ffffff;
         --surface: #ffffff;
         --success: #067647;
         --warning: #b54708;
         --danger: #b42318;
      }

      * { box-sizing: border-box; }

      body {
         margin: 0;
         padding: 48px 20px 36px;
         background: var(--background);
         color: var(--ink);
         font-family: "Manrope", ui-sans-serif, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
         -webkit-font-smoothing: antialiased;
      }

      button, input { font: inherit; }

      .container {
         width: 100%;
         max-width: 920px;
         margin: 0 auto;
      }

      .page-header {
         margin-bottom: 32px;
         text-align: center;
      }

      .brand-icon {
         display: grid;
         width: 46px;
         height: 46px;
         margin: 0 auto 16px;
         place-items: center;
         border-radius: 13px;
```

```
    background: var(--primary);
    color: white;
    box-shadow: 0 8px 20px rgba(37, 99, 235, 0.18);
}

.brand-icon svg {
    width: 23px;
    height: 23px;
}

h1 {
    margin: 0 0 8px;
    font-size: clamp(28px, 5vw, 35px);
    font-weight: 800;
    letter-spacing: -0.04em;
}

.page-header p {
    margin: 0;
    color: var(--muted);
    font-size: 13px;
    line-height: 1.6;
}

.wallet {
    display: flex;
    align-items: center;
    gap: 18px;
    margin-bottom: 18px;
    padding: 15px 16px;
    border: 1px solid var(--line);
    border-radius: 12px;
    background: var(--surface);
    box-shadow: 0 2px 8px rgba(16, 24, 40, 0.035);
}

.wallet-info {
    min-width: 0;
    flex: 1;
}

.wallet-info small {
    display: block;
    margin-bottom: 4px;
    color: var(--muted);
    font-size: 12px;
}

#userAddress {
    display: block;
    overflow: hidden;
    font-family: ui-monospace, SFMono-Regular, Menlo, monospace;
    font-size: 13px;
    font-weight: 600;
    text-overflow: ellipsis;
    white-space: nowrap;
}

button {
    min-height: 42px;
```

```
    padding: 0 19px;
    border: 0;
    border-radius: 8px;
    background: var(--primary);
    color: white;
    cursor: pointer;
    font-size: 13px;
    font-weight: 700;
    transition: background 160ms ease, transform 160ms ease;
}

button:hover {
    background: var(--primary-dark);
    transform: translateY(-1px);
}

button:disabled {
    cursor: not-allowed;
    opacity: .7;
    transform: none;
}

button[aria-busy="true"] { cursor: wait; }

button.is-complete,
button.is-complete:hover {
    border-color: #d7d7d7;
    background: #f3f3f3;
    color: #303030;
    opacity: 1;
}

.button-spinner {
    display: inline-block;
    width: 14px;
    height: 14px;
    margin-right: 8px;
    border: 2px solid rgba(255, 255, 255, .38);
    border-top-color: #ffffff;
    border-radius: 50%;
    vertical-align: -2px;
    animation: button-spin .7s linear infinite;
}

@keyframes button-spin {
    to { transform: rotate(360deg); }
}

button:active { transform: none; }

button.secondary {
    border: 1px solid var(--line);
    background: white;
    color: var(--ink);
}

button.secondary:hover { background: #f9fafb; }

.section {
    margin-bottom: 16px;
```

```
    padding: 23px;
    border: 1px solid var(--line);
    border-radius: 12px;
    background: var(--surface);
    box-shadow: 0 2px 8px rgba(16, 24, 40, 0.035);
}

.section-head {
    display: flex;
    align-items: flex-start;
    justify-content: space-between;
    gap: 16px;
    margin-bottom: 18px;
}

.section h2 {
    margin: 0 0 5px;
    font-size: 17px;
    font-weight: 750;
    letter-spacing: -0.02em;
}

.section p {
    margin: 0;
    color: var(--muted);
    font-size: 12px;
    line-height: 1.55;
}

.step {
    display: inline-grid;
    width: 28px;
    height: 28px;
    flex: 0 0 auto;
    place-items: center;
    border-radius: 7px;
    background: #eef4ff;
    color: var(--primary);
    font-size: 12px;
    font-weight: 750;
}

.title-row {
    display: flex;
    align-items: flex-start;
    gap: 14px;
}

.balance-box {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 12px;
}

.balance-card {
    padding: 19px 20px;
    border: 1px solid #dbe7ff;
    border-radius: 10px;
    background: #f7faff;
    color: #475467;
```

```
    font-size: 12px;
}

.balance-card strong {
    display: block;
    margin-top: 8px;
    color: #173c86;
    font-size: clamp(22px, 4vw, 28px);
    font-weight: 750;
    letter-spacing: -0.03em;
}

.grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 16px;
}

.grid .section { margin-bottom: 0; }

.grid + .section,
.section + .grid { margin-top: 16px; }

label {
    display: block;
    margin: 0 0 7px;
    color: #344054;
    font-size: 12px;
    font-weight: 650;
}

input {
    width: 100%;
    height: 43px;
    margin-bottom: 12px;
    padding: 0 13px;
    border: 1px solid #d0d5dd;
    border-radius: 8px;
    background: #ffffff;
    color: var(--ink);
    font-size: 13px;
    outline: none;
    transition: border 160ms ease, box-shadow 160ms ease;
}

input::placeholder { color: #98a2b3; }

input:focus {
    border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.11);
}

.form-row {
    display: grid;
    grid-template-columns: minmax(0, 1fr) 180px;
    gap: 12px;
}

.form-actions { margin-top: 2px; }
```

```
.info {
  margin-top: 14px;
  padding: 13px 14px;
  border-radius: 9px;
  background: #f8f9fb;
  color: #475467;
  font-size: 13px;
  word-break: break-all;
}

#accountsList {
  display: grid;
  gap: 8px;
  margin-top: 14px;
}

.account {
  display: flex;
  min-width: 0;
  align-items: center;
  gap: 10px;
  padding: 11px 12px;
  border-radius: 9px;
  background: #f8f9fb;
}

.account-index {
  display: grid;
  width: 25px;
  height: 25px;
  flex: 0 0 auto;
  place-items: center;
  border-radius: 7px;
  background: #dbeafe;
  color: var(--primary-dark);
  font-size: 10px;
  font-weight: 750;
}

.account-address {
  overflow: hidden;
  font-family: ui-monospace, SFMono-Regular, Menlo, monospace;
  font-size: 11px;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.empty-state {
  padding: 13px;
  border-radius: 9px;
  background: #f9fafb;
  color: var(--muted);
  font-size: 12px;
  text-align: center;
}

#status {
  margin-top: 18px;
  padding: 13px 16px;
  border: 1px solid #fedf89;
```

```
    border-radius: 9px;
    background: #fffaeb;
    color: var(--warning);
    font-size: 13px;
    font-weight: 650;
    text-align: center;
    transition: 160ms ease;
}

#status.visible { box-shadow: 0 4px 16px rgba(181, 71, 8, 0.08); }
#status.error { border-color: #fecdca; background: #fef3f2; color: var(--danger); }

.footer {
    margin: 24px 0 0;
    color: #98a2b3;
    font-size: 12px;
    text-align: center;
}

@media (max-width: 680px) {
    body { padding: 28px 14px; }
    .wallet { align-items: stretch; flex-direction: column; }
    .wallet button { width: 100%; }
    .grid, .balance-box, .form-row { grid-template-columns: 1fr; }
    .section { padding: 20px; }
    .section-head { align-items: stretch; flex-direction: column; }
    .section-head > button { width: 100%; }
}

/* Simple monochrome card system based on the supplied reference flow. */
:root {
    --primary: #111111;
    --primary-dark: #000000;
    --primary-soft: #f4f4f4;
    --ink: #111111;
    --muted: #6b6b6b;
    --line: #dddddd;
    --background: #f5f5f5;
    --surface: #ffffff;
}

body {
    padding: 32px 20px 48px;
    background: var(--background);
    font-family: "Geist", ui-sans-serif, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
}

.container { max-width: 940px; }

.site-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 20px;
    margin-bottom: 20px;
    padding: 4px 4px 10px;
}

.wordmark {
    color: var(--ink);
```

```
    font-size: 25px;
    font-weight: 650;
    letter-spacing: -0.045em;
    text-decoration: none;
}

.network-label { color: var(--muted); font-size: 13px; font-weight: 500; }
.site-actions { display: flex; align-items: center; gap: 16px; }

button {
    min-height: 44px;
    padding: 0 18px;
    border: 1px solid #111111;
    border-radius: 9px;
    background: #111111;
    color: #fff;
    font-size: 14px;
    font-weight: 600;
}

button:hover {
    background: #000;
    transform: none;
}

button.secondary {
    border-color: #111111;
    background: #111111;
    color: #ffffff;
}

button.secondary:hover { background: #000000; }

.wallet {
    margin: 0 0 20px;
    padding: 15px 18px;
    border: 1px solid var(--line);
    border-radius: 12px;
    background: #ffffff;
    box-shadow: 0 5px 18px rgba(0, 0, 0, .035);
}

.wallet-info {
    display: flex;
    align-items: baseline;
    justify-content: space-between;
    gap: 24px;
}

.wallet-info small {
    margin: 0;
    color: var(--ink);
    font-size: 14px;
    font-weight: 500;
}

#userAddress {
    color: var(--muted);
    font-family: "Geist Mono", ui-monospace, monospace;
    font-size: 12px;
```

```
    font-weight: 400;
}

.account-state {
    display: flex;
    max-width: 330px;
    align-items: flex-start;
    gap: 10px;
    padding-left: 18px;
    border-left: 1px solid var(--line);
}

.state-dot {
    width: 9px;
    height: 9px;
    margin-top: 5px;
    flex: 0 0 auto;
    border-radius: 50%;
    background: #a3a3a3;
}

.account-state strong,
.account-state small { display: block; }

.account-state strong {
    margin-bottom: 3px;
    font-size: 13px;
    font-weight: 650;
}

.account-state small {
    color: var(--muted);
    font-size: 11px;
    line-height: 1.45;
}

.account-state[data-state="registered"] .state-dot { background: #16803c; }
.account-state[data-state="required"] .state-dot { background: #b76e00; }
.account-state[data-state="checking"] .state-dot {
    background: #111111;
    animation: state-pulse 1s ease-in-out infinite alternate;
}

@keyframes state-pulse { to { opacity: .3; } }

.section {
    margin: 0 0 20px;
    padding: 32px;
    border: 1px solid #e3e3e3;
    border-radius: 18px;
    background: #ffffff;
    box-shadow: 0 8px 24px rgba(0, 0, 0, .045);
}

.section-head {
    align-items: flex-start;
    flex-direction: column;
    margin-bottom: 24px;
}
```

```
.section h2 {
    margin-bottom: 8px;
    font-size: clamp(23px, 4vw, 30px);
    font-weight: 650;
    letter-spacing: -0.04em;
}

.section p {
    color: var(--muted);
    font-size: 14px;
    line-height: 1.55;
}

.step {
    display: inline;
    width: auto;
    height: auto;
    border-radius: 0;
    background: transparent;
    color: var(--ink);
    font-size: clamp(23px, 4vw, 30px);
    font-weight: 650;
    letter-spacing: -0.04em;
}

.step::after { content: "."; }
.title-row { align-items: baseline; gap: 8px; }

.balance-box {
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 14px;
    border: 0;
}

.balance-card {
    padding: 24px;
    border: 1px solid var(--line);
    border-radius: 12px;
    background: #fafafa;
    color: var(--muted);
    font-size: 14px;
}

.balance-card + .balance-card {
    margin-left: 0;
    padding-left: 24px;
    border-left: 1px solid var(--line);
}

.balance-card strong {
    margin-top: 12px;
    color: var(--ink);
    font-size: clamp(28px, 5vw, 40px);
    font-weight: 650;
    letter-spacing: -0.04em;
}

.grid {
    display: block;
    margin: 0 !important;
```

```
    border: 0;
}

.grid .section { margin-bottom: 20px; }
.grid .section:first-child { padding-right: 32px; border-right: 1px solid #e3e3e3; }
.grid .section:last-child { padding-left: 32px; }

label {
    margin-bottom: 9px;
    color: var(--ink);
    font-size: 13px;
    font-weight: 500;
}

input {
    height: 52px;
    margin-bottom: 14px;
    border-color: #cfcfcf;
    border-radius: 9px;
    font-size: 15px;
}

input:focus {
    border-color: #1d1d1f;
    box-shadow: 0 0 0 1px #1d1d1f;
}

input.input-error,
input.input-error:focus {
    border-color: #b42318;
    box-shadow: 0 0 0 1px #b42318;
}

.field-error {
    margin: -7px 0 12px;
    color: #b42318;
    font-size: 12px;
    line-height: 1.4;
}

.info {
    margin-top: 18px;
    padding: 14px 16px;
    border-radius: 9px;
    background: #f5f5f5;
    color: var(--muted);
    font-size: 13px;
}

#accountsList { gap: 10px; margin-top: 22px; }

.account, .empty-state { border-radius: 12px; }

.account {
    padding: 14px 16px;
    border: 1px solid var(--line);
    background: #ffffff;
}

.account-index {
```

```
    width: 38px;
    height: 38px;
    border-radius: 10px;
    background: #111111;
    color: #ffffff;
    font-family: "Geist", ui-sans-serif, sans-serif;
    font-size: 12px;
    font-weight: 600;
}

.account-details {
    min-width: 0;
    flex: 1;
}

.account-name {
    margin-bottom: 4px;
    color: var(--ink);
    font-size: 14px;
    font-weight: 600;
}

.account-address {
    display: block;
    color: var(--muted);
    font-family: "Geist Mono", ui-monospace, monospace;
    font-size: 11px;
}

.account-badge {
    flex: 0 0 auto;
    padding: 5px 8px;
    border-radius: 999px;
    background: #ecfdf3;
    color: #067647;
    font-size: 10px;
    font-weight: 600;
}

.empty-state {
    padding: 15px;
    border: 0;
    text-align: left;
}

#status {
    margin: 0 0 20px;
    padding: 14px 16px;
    border: 1px solid var(--line);
    border-radius: 10px;
    background: #ffffff;
    color: var(--muted);
    font-size: 13px;
    font-weight: 400;
    text-align: center;
}

#status.visible { box-shadow: none; }
#status.success { border-color: #abefc6; background: #ecfdf3; color: #067647; }
#status.error { border-color: #fecdca; background: #fef3f2; color: #b42318; }
```

```
      .footer {
        margin: 28px 0 0;
        color: var(--muted);
        text-align: center;
      }

      @media (max-width: 760px) {
        body { padding: 20px 14px 32px; }
        .site-header { align-items: flex-start; flex-direction: column; }
        .site-actions { width: 100%; justify-content: space-between; }
        .site-header button { min-height: 40px; padding: 0 15px; }
        .wallet { align-items: initial; flex-direction: row; }
        .wallet-info { align-items: flex-start; flex-direction: column; gap: 8px; }
        .account-state { max-width: none; padding: 14px 0 0; border-top: 1px solid var(--line); border-left: 0;
}
        .wallet { flex-direction: column; }
        .section { padding: 24px 20px; }
        .section-head { align-items: flex-start; }
        .section-head > button { width: 100%; }
        .grid, .balance-box, .form-row { grid-template-columns: 1fr; }
        .balance-card + .balance-card { margin: 0; padding-left: 24px; border-left: 1px solid var(--line); }
        .grid .section:first-child,
        .grid .section:last-child { padding: 24px 20px; border-right: 1px solid #e3e3e3; }
        .account { align-items: flex-start; }
        .account-badge { margin-top: 2px; }
      }
    </style>
  </head>
  <body>
    <div class="container">
      <header class="site-header">
        <a class="wordmark" href="#">Simple Banking</a>
        <div class="site-actions">
          <span class="network-label">Sepolia Testnet</span>
          <button id="connectBtn" type="button">Connect MetaMask</button>
        </div>
      </header>

      <div class="wallet">
        <div class="wallet-info" id="walletIdentity">
          <small>Connected account</small>
          <span id="userAddress">Not connected</span>
        </div>
        <div class="account-state" id="accountState" data-state="disconnected" aria-live="polite">
          <span class="state-dot" aria-hidden="true"></span>
          <div>
            <strong id="registrationTitle">Wallet not connected</strong>
            <small id="registrationText">Connect MetaMask to check your banking account.</small>
          </div>
        </div>
      </div>

      <section class="section" id="overview">
        <div class="section-head">
          <div>
            <h2>My Dashboard</h2>
          </div>
          <button type="button" onclick="refreshDashboard(true, this)">Refresh Balances</button>
        </div>
```

```
        <div class="balance-box">
          <div class="balance-card">Bank Tokens (Bonus)<strong><span id="tokenBalance">0</span>
BANK</strong></div>
          <div class="balance-card">Deposited ETH<strong id="ethBalance">0 ETH</strong></div>
        </div>
      </section>

      <section class="section">
        <div class="section-head">
          <div class="title-row"><span class="step">1</span><div><h2>Register New
Account</h2><p>New accounts receive 100 BANK tokens instantly.</p></div></div>
          <button id="registerBtn" type="button" onclick="registerAccount(this)" disabled>Connect Wallet
First</button>
        </div>
      </section>

      <div class="grid" id="transactions">
        <section class="section">
          <div class="title-row"><span class="step">2</span><div><h2>Deposit ETH</h2><p>Add ETH
to your bank balance.</p></div></div>
          <div style="margin-top:18px"><label for="depositAmount">Amount in ETH</label><input
type="number" id="depositAmount" placeholder="e.g. 0.01" min="0" step="0.0001" inputmode="decimal"
required><button type="button" onclick="deposit(this)">Deposit ETH</button></div>
        </section>
        <section class="section">
          <div class="title-row"><span class="step">3</span><div><h2>Withdraw
ETH</h2><p>Withdraw ETH to your wallet.</p></div></div>
          <div style="margin-top:18px"><label for="withdrawAmount">Amount in ETH</label><input
type="number" id="withdrawAmount" placeholder="e.g. 0.01" min="0" step="0.0001"
inputmode="decimal" required><button class="secondary" type="button"
onclick="withdraw(this)">Withdraw ETH</button></div>
        </section>
      </div>

      <section class="section">
        <div class="title-row"><span class="step">4</span><div><h2>Transfer Bank Tokens</h2><p>Send
tokens to another registered account.</p></div></div>
        <div class="form-row" style="margin-top:18px">
          <div><label for="recipient">Recipient wallet address</label><input type="text" id="recipient"
placeholder="0x..." autocomplete="off" spellcheck="false" required></div>
          <div><label for="transferAmount">Number of tokens</label><input type="number"
id="transferAmount" placeholder="0" min="1" step="1" inputmode="numeric" required></div>
        </div>
        <div class="form-actions"><button type="button" onclick="transfer(this)">Transfer
Tokens</button></div>
      </section>

      <div class="grid" id="accounts">
        <section class="section">
          <div class="title-row"><span class="step">5</span><div><h2>Registered
Accounts</h2><p>View all bank members.</p></div></div>
          <button type="button" style="margin-top:18px" onclick="loadAccounts(true, this)">Load
Accounts</button>
          <div id="accountsList"><div class="empty-state">No accounts loaded yet.</div></div>
        </section>
        <section class="section">
          <div class="title-row"><span class="step">6</span><div><h2>Contract Liquidity</h2><p>Check
the bank's ETH reserve.</p></div></div>
          <button type="button" style="margin-top:18px" onclick="checkContractBalance(true,
this)">Check ETH Reserve</button>
```

```
        <div class="info">Contract Balance: <strong id="contractBalance">0</strong> ETH</div>
      </section>
    </div>

    <div id="status" class="error" role="status" aria-live="polite">Wallet not connected.</div>
    <p class="footer">Powered by Ethereum · Sepolia Testnet · MetaMask</p>
  </div>

  <script>
    // Replace with your Remix Contract Address after deployment
    const CONTRACT_ADDRESS = "0xF9C7a986D919DE5603443C6794369D5d5769C0dB";

    const CONTRACT_ABI = [
      "function registernewaccount() external",
      "function deposit() external payable",
      "function withdraw(uint256 amount) external",
      "function transfer(address to, uint256 amount) external",
      "function getEthBalance(address user) external view returns (uint256)",
      "function getTokenBalance(address user) external view returns (uint256)",
      "function getAccounts() external view returns (address[])",
      "function contractBalance() external view returns (uint256)",
      "function registered(address) external view returns (bool)"
    ];

    let provider;
    let signer;
    let contract;
    let statusTimer;

    document.getElementById("connectBtn").onclick = connectWallet;

    async function connectWallet() {
      const button = document.getElementById("connectBtn");
      let stopLoading;
      try {
        if (!window.ethereum) {
          alert("Please install MetaMask.");
          return;
        }

        try {
          await window.ethereum.request({
            method: "wallet_switchEthereumChain",
            params: [{ chainId: "0xaa36a7" }]
          });
        } catch (switchError) {
          console.log("Could not switch to Sepolia automatically: ", switchError);
        }

        if (CONTRACT_ADDRESS === "YOUR_CONTRACT_ADDRESS_HERE" ||
!CONTRACT_ADDRESS) {
          alert("Please paste your deployed contract address in index.html (CONTRACT_ADDRESS).");
          return;
        }

        stopLoading = setButtonLoading(button, "Connecting...");
        showStatus("Connecting to MetaMask...");
        provider = new ethers.BrowserProvider(window.ethereum);
        await provider.send("eth_requestAccounts", []);
        signer = await provider.getSigner();
```

```
        contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI, signer);

        const address = await signer.getAddress();
        updateWalletUI(address);
        showStatus("Wallet connected successfully.", false, true);
        await refreshDashboard(false);
    } catch (error) {
        console.error(error);
        showStatus("Wallet connection failed.", true);
    } finally {
        if (stopLoading) stopLoading(button.dataset.connectedLabel);
    }
}

async function registerAccount(button) {
    if (!contract) return showStatus("Connect MetaMask first.", true);
    const stopLoading = setButtonLoading(button, "Registering...");
    try {
        showStatus("Confirm account registration in MetaMask...");
        const tx = await contract.registernewaccount();
        showStatus("Registration submitted. Waiting for confirmation...");
        await tx.wait();
        showStatus("Account registered! You received 100 Bank Tokens.");
        await checkBalance();
        await checkRegistration();
        await loadAccounts(false);
    } catch (error) {
        showStatus(getErrorMessage(error), true);
    } finally {
        stopLoading();
    }
}

async function deposit(button) {
    if (!contract) return showStatus("Connect MetaMask first.", true);
    const input = document.getElementById("depositAmount");
    const amount = input.value.trim();
    clearFieldError(input);
    if (!amount) {
        showFieldError(input, "Enter an ETH amount.");
        return showStatus("Deposit amount is required.", true);
    }
    if (!Number.isFinite(Number(amount)) || Number(amount) <= 0) {
        showFieldError(input, "Amount must be greater than 0.");
        return showStatus("Enter a valid ETH amount.", true);
    }
    let amountInWei;
    try {
        amountInWei = ethers.parseEther(amount);
    } catch {
        showFieldError(input, "Enter a valid ETH amount with up to 18 decimals.");
        return showStatus("Enter a valid ETH amount.", true);
    }
    const stopLoading = setButtonLoading(button, "Depositing...");
    try {
        showStatus("Confirm your deposit in MetaMask...");
        const tx = await contract.deposit({ value: amountInWei });
        showStatus("Deposit submitted. Waiting for confirmation...");
        await tx.wait();
        showStatus("Deposit successful.");
```

```
        document.getElementById("depositAmount").value = "";
        await checkBalance();
        await checkContractBalance();
    } catch (error) {
        showStatus(getErrorMessage(error), true);
    } finally {
        stopLoading();
    }
}

async function withdraw(button) {
    if (!contract) return showStatus("Connect MetaMask first.", true);
    const input = document.getElementById("withdrawAmount");
    const amount = input.value.trim();
    clearFieldError(input);
    if (!amount) {
        showFieldError(input, "Enter an ETH amount.");
        return showStatus("Withdrawal amount is required.", true);
    }
    if (!Number.isFinite(Number(amount)) || Number(amount) <= 0) {
        showFieldError(input, "Amount must be greater than 0.");
        return showStatus("Enter a valid ETH amount.", true);
    }
    let amountInWei;
    try {
        amountInWei = ethers.parseEther(amount);
    } catch {
        showFieldError(input, "Enter a valid ETH amount with up to 18 decimals.");
        return showStatus("Enter a valid ETH amount.", true);
    }
    const stopLoading = setButtonLoading(button, "Withdrawing...");
    try {
        showStatus("Confirm your withdrawal in MetaMask...");
        const tx = await contract.withdraw(amountInWei);
        showStatus("Withdrawal submitted. Waiting for confirmation...");
        await tx.wait();
        showStatus("Withdrawal successful.");
        document.getElementById("withdrawAmount").value = "";
        await checkBalance();
        await checkContractBalance();
    } catch (error) {
        showStatus(getErrorMessage(error), true);
    } finally {
        stopLoading();
    }
}

async function transfer(button) {
    if (!contract) return showStatus("Connect MetaMask first.", true);
    const recipientInput = document.getElementById("recipient");
    const amountInput = document.getElementById("transferAmount");
    const recipient = recipientInput.value.trim();
    const amount = amountInput.value.trim();
    clearFieldError(recipientInput);
    clearFieldError(amountInput);

    let firstInvalidField = null;
    if (!recipient) {
        showFieldError(recipientInput, "Enter the recipient wallet address.", false);
        firstInvalidField = recipientInput;
```

```
        } else if (!ethers.isAddress(recipient)) {
            showFieldError(recipientInput, "Enter a valid Ethereum address.", false);
            firstInvalidField = recipientInput;
        } else if (recipient.toLowerCase() === ethers.ZeroAddress.toLowerCase()) {
            showFieldError(recipientInput, "The zero address cannot receive tokens.", false);
            firstInvalidField = recipientInput;
        }

        if (!amount) {
            showFieldError(amountInput, "Enter the number of tokens.", false);
            firstInvalidField ||= amountInput;
        } else if (!Number.isFinite(Number(amount)) || Number(amount) <= 0 ||
!Number.isInteger(Number(amount))) {
            showFieldError(amountInput, "Enter a whole number greater than 0.", false);
            firstInvalidField ||= amountInput;
        }

        if (firstInvalidField) {
            firstInvalidField.focus();
            return showStatus("Complete the highlighted transfer fields.", true);
        }
        const stopLoading = setButtonLoading(button, "Transferring...");
        try {
            showStatus("Confirm your token transfer in MetaMask...");
            const tx = await contract.transfer(recipient, amount);
            showStatus("Transfer submitted. Waiting for confirmation...");
            await tx.wait();
            showStatus("Transfer successful.");
            document.getElementById("recipient").value = "";
            document.getElementById("transferAmount").value = "";
            await checkBalance();
        } catch (error) {
            showStatus(getErrorMessage(error), true);
        } finally {
            stopLoading();
        }
    }

    async function checkBalance() {
        if (!contract || !signer) return;
        try {
            const address = await signer.getAddress();
            const [tokenBalance, ethBalance] = await Promise.all([
                contract.getTokenBalance(address),
                contract.getEthBalance(address)
            ]);
            document.getElementById("tokenBalance").innerText = tokenBalance.toString();
            document.getElementById("ethBalance").innerText = formatEth(ethBalance) + " ETH";
        } catch (error) {
            console.error(error);
        }
    }

    async function loadAccounts(announce = true, button = null) {
        if (!contract) return showStatus("Connect MetaMask first.", true);
        const stopLoading = setButtonLoading(button, "Loading...");
        try {
            if (announce) showStatus("Loading registered accounts...");
            const accounts = await contract.getAccounts();
            const currentAddress = signer ? (await signer.getAddress()).toLowerCase() : "";
```

```
            const list = document.getElementById("accountsList");
            list.innerHTML = "";

            if (accounts.length === 0) {
                list.innerHTML = '<div class="empty-state">No accounts registered yet.</div>';
                if (announce) showStatus("No registered accounts found.");
                return;
            }

            accounts.forEach((account, index) => {
                const row = document.createElement("div");
                row.className = "account";
                const number = document.createElement("span");
                number.className = "account-index";
                number.textContent = `A${index + 1}`;

                const details = document.createElement("div");
                details.className = "account-details";
                const name = document.createElement("div");
                name.className = "account-name";
                name.textContent = `Account ${index + 1}`;
                const address = document.createElement("span");
                address.className = "account-address";
                address.title = account;
                address.textContent = account;
                details.append(name, address);
                row.append(number, details);

                if (account.toLowerCase() === currentAddress) {
                    const badge = document.createElement("span");
                    badge.className = "account-badge";
                    badge.textContent = "Current";
                    row.appendChild(badge);
                }

                list.appendChild(row);
            });

            if (announce) showStatus(`accounts.lengthregisteredaccount{accounts.length === 1 ? "" :
"s"} loaded.`);
        } catch (error) {
            showStatus("Unable to load accounts.", true);
        } finally {
            stopLoading();
        }
    }

    async function checkContractBalance(announce = false, button = null) {
        if (!contract) {
            if (announce) showStatus("Connect MetaMask first.", true);
            return;
        }
        const stopLoading = setButtonLoading(button, "Checking...");
        try {
            const balance = await contract.contractBalance();
            document.getElementById("contractBalance").innerText = formatEth(balance);
            if (announce) showStatus("Bank reserve updated.");
        } catch (error) {
            console.error(error);
            if (announce) showStatus("Unable to check the bank reserve.", true);
```

```
    } finally {
        stopLoading();
    }
}

async function checkRegistration() {
    if (!contract || !signer) return;
    try {
        const address = await signer.getAddress();
        const isRegistered = await contract.registered(address);
        setRegistrationState(isRegistered ? "registered" : "required");
    } catch (error) {
        console.error(error);
        setRegistrationState("unknown");
    }
}

async function refreshDashboard(announce = true, button = null) {
    if (!contract) return showStatus("Connect MetaMask first.", true);
    const stopLoading = setButtonLoading(button, "Refreshing...");
    try {
        await Promise.all([checkBalance(), checkContractBalance(), checkRegistration()]);
        if (announce) showStatus("Balances refreshed.");
    } finally {
        stopLoading();
    }
}

function updateWalletUI(address) {
    document.getElementById("userAddress").innerText = address;
    const button = document.getElementById("connectBtn");
    const shortAddress = `address.slice(0, 6)...{address.slice(-4)}`;
    button.dataset.connectedLabel = shortAddress;
    if (!button.disabled) button.textContent = shortAddress;
    setRegistrationState("checking");
}

function setRegistrationState(state) {
    const container = document.getElementById("accountState");
    const title = document.getElementById("registrationTitle");
    const text = document.getElementById("registrationText");
    const registerButton = document.getElementById("registerBtn");
    container.dataset.state = state;

    if (state === "registered") {
        title.textContent = "Registered account";
        text.textContent = "This wallet can deposit, withdraw, and transfer BANK.";
        registerButton.dataset.locked = "true";
        registerButton.dataset.lockedLabel = "✓ Account Registered";
        registerButton.classList.add("is-complete");
        registerButton.disabled = true;
        if (!registerButton.hasAttribute("aria-busy")) {
            registerButton.textContent = registerButton.dataset.lockedLabel;
        }
        return;
    }

    delete registerButton.dataset.locked;
    delete registerButton.dataset.lockedLabel;
    registerButton.classList.remove("is-complete");
```

```
    if (state === "required") {
        title.textContent = "Registration required";
        text.textContent = "Register this wallet to receive 100 BANK tokens.";
        registerButton.disabled = false;
        registerButton.textContent = "Register Account";
    } else if (state === "checking") {
        title.textContent = "Checking account";
        text.textContent = "Reading registration status from Sepolia.";
        registerButton.disabled = true;
        registerButton.textContent = "Checking Account...";
    } else {
        title.textContent = "Status unavailable";
        text.textContent = "Refresh the dashboard to check this account again.";
        registerButton.disabled = false;
        registerButton.textContent = "Register Account";
    }
}

function setButtonLoading(button, loadingText = "Processing...") {
    if (!button) return () => {};
    const originalHtml = button.innerHTML;
    button.disabled = true;
    button.setAttribute("aria-busy", "true");
    button.innerHTML = `<span class="button-spinner" aria-hidden="true"></span>${loadingText}`;

    return function stopLoading(nextLabel = "") {
        button.removeAttribute("aria-busy");
        if (button.dataset.locked === "true") {
            button.disabled = true;
            button.textContent = button.dataset.lockedLabel || "Completed";
        } else if (nextLabel) {
            button.disabled = false;
            button.textContent = nextLabel;
        } else {
            button.disabled = false;
            button.innerHTML = originalHtml;
        }
    };
}

function showFieldError(input, message, focus = true) {
    clearFieldError(input);
    input.classList.add("input-error");
    input.setAttribute("aria-invalid", "true");

    const error = document.createElement("div");
    error.className = "field-error";
    error.id = `${input.id}Error`;
    error.setAttribute("role", "alert");
    error.textContent = message;
    input.setAttribute("aria-describedby", error.id);
    input.insertAdjacentElement("afterend", error);

    if (focus) input.focus();
}

function clearFieldError(input) {
    input.classList.remove("input-error");
    input.removeAttribute("aria-invalid");
```

```
        const errorId = input.getAttribute("aria-describedby");
        if (errorId) document.getElementById(errorId)?.remove();
        input.removeAttribute("aria-describedby");
      }

      document.querySelectorAll("input").forEach((input) => {
        input.addEventListener("input", () => clearFieldError(input));
      });

      function formatEth(value) {
        const formatted = ethers.formatEther(value);
        const numeric = Number(formatted);
        if (!Number.isFinite(numeric)) return formatted;
        if (numeric === 0) return "0";
        if (numeric < 0.0001) return "<0.0001";
        return numeric.toLocaleString(undefined, { maximumFractionDigits: 4, minimumFractionDigits: 0
});
      }

      function getErrorMessage(error) {
        if (error.reason) return error.reason;
        if (error.shortMessage) return error.shortMessage;
        if (error.message) {
          if (error.message.toLowerCase().includes("user rejected")) return "Transaction rejected by user.";
          return error.message;
        }
        return "Transaction failed.";
      }

      function showStatus(message, isError = false, isSuccess = false) {
        const status = document.getElementById("status");
        status.innerText = message;
        status.classList.toggle("error", isError);
        status.classList.toggle("success", isSuccess && !isError);
        status.classList.add("visible");
        clearTimeout(statusTimer);
        statusTimer = setTimeout(() => status.classList.remove("visible"), 5200);
      }

      if (window.ethereum) {
        window.ethereum.on("accountsChanged", async function (accounts) {
          if (!provider) return;
          if (accounts.length === 0) {
            window.location.reload();
            return;
          }
          signer = await provider.getSigner();
          contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI, signer);
          const address = await signer.getAddress();
          updateWalletUI(address);
          await refreshDashboard();
        });

        window.ethereum.on("chainChanged", function () {
          window.location.reload();
        });
      }
    </script>
  </body>
</html
```

2. Test Cases:

TC ID	Test Case	Preconditions	Test Steps	Expected Result	Status
TC01	Wallet Connection & Network Switch	MetaMask installed; DApp open on local server	1. Click "Connect MetaMask" 2. Confirm connection in popup	Wallet connects; UI switches to Sepolia; address displayed	**PASS**
TC02	New Account Registration	Wallet connected; account not registered	1. Click "Register Account" 2. Confirm transaction in MetaMask 3. Click "Check Balance"	Transaction mined; user receives 100 Bank Tokens	**PASS**
TC03	Duplicate Registration Prevention	Account already registered	1. Click "Register Account" again	Transaction reverts with "Account already registered"	**PASS**
TC04	Deposit Native ETH	Account registered; wallet holds Sepolia ETH	1. Enter 0.01 in Deposit field 2. Click "Deposit" 3. Confirm in MetaMask	0.01 ETH transferred to contract; dashboard shows 0.01 ETH	**PASS**
TC05	Withdraw Native ETH	Account has deposited ETH in contract	1. Enter 0.01 in Withdraw field 2. Click "Withdraw" 3. Confirm in MetaMask	0.01 ETH returned to wallet; dashboard shows 0.0 ETH	**PASS**
TC06	Over-withdrawal Prevention	Deposited ETH balance is 0.0 ETH	1. Enter 1.0 in Withdraw field	Transaction reverts	**PASS**

			2. Click "Withdraw"	with "Insufficient ETH balance"	
TC07	Peer-to-Peer Token Transfer	Both accounts registered; Sender has ≥ 10 tokens	1. Enter Recipient address 2. Enter 10 tokens 3. Click "Transfer"	Sender tokens: 100 → 90; Receiver tokens: 100 → 110	**PASS**
TC08	Transfer to Unregistered Account	Sender registered; Recipient not registered	1. Enter unregistered address 2. Enter 5 tokens 3. Click "Transfer"	Transaction reverts with "Recipient not registered"	**PASS**
TC09	Check Account Balance (balanceOf)	Wallet connected	1. Click "Check Balance" button	Displays current tokens and deposited ETH with 0 gas cost	**PASS**
TC10	List All Registered Accounts	At least one account registered	1. Click "List Accounts" button	Displays all registered user addresses dynamically on screen	**PASS**

Output:

TC01:

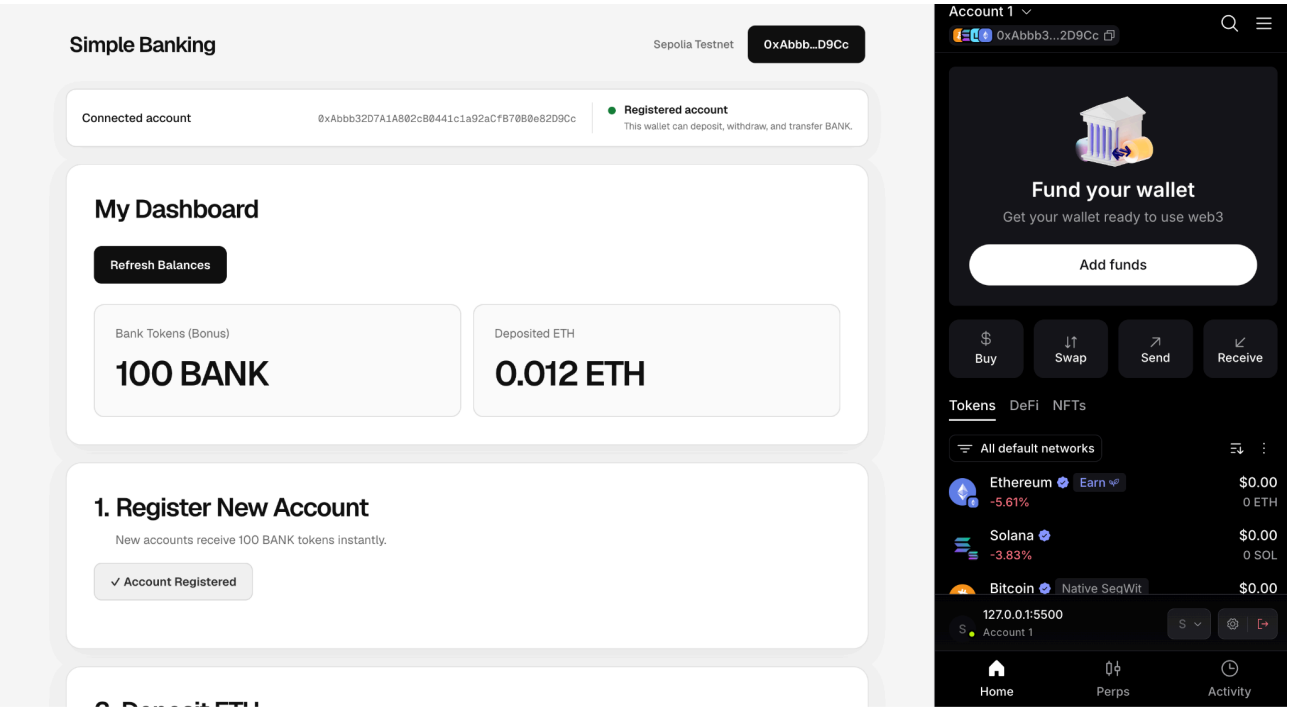


Figure 1: Successful connection of MetaMask wallet to the DApp on Sepolia Testnet.

TC02:

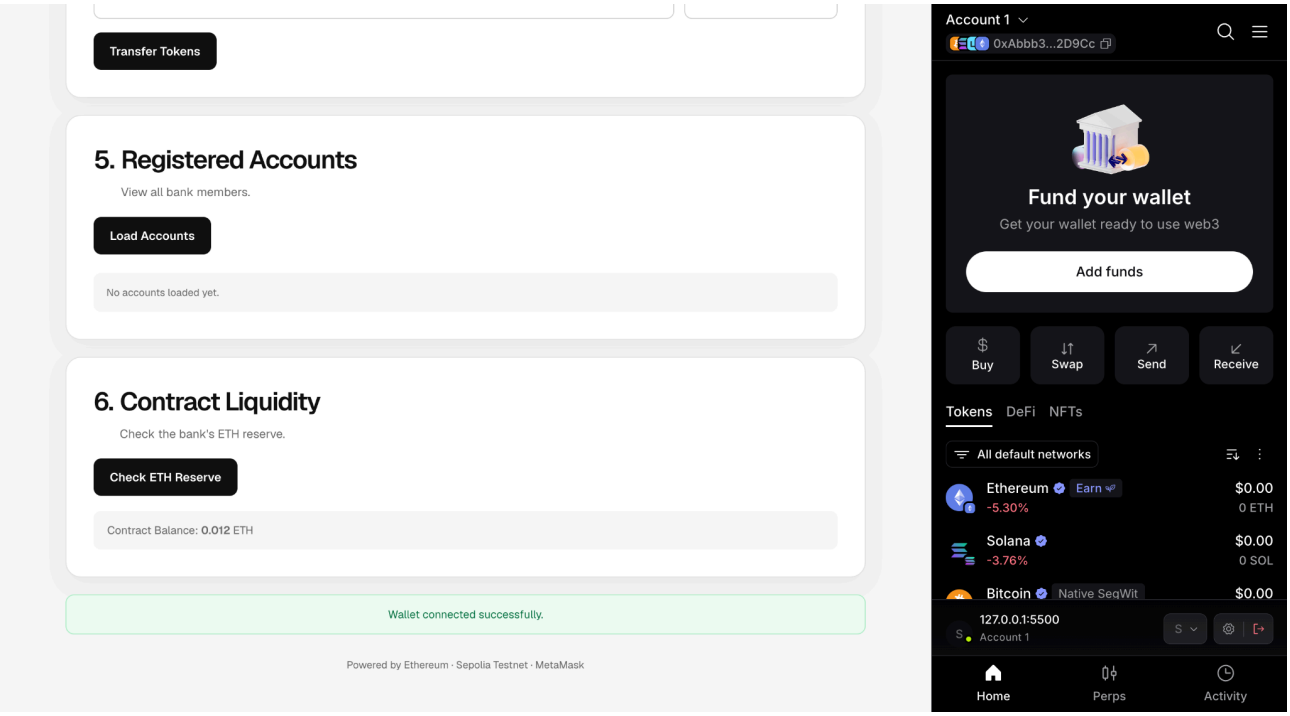


Figure 2: New account registration confirming receipt of initial 100 Bank Tokens.

TC03:

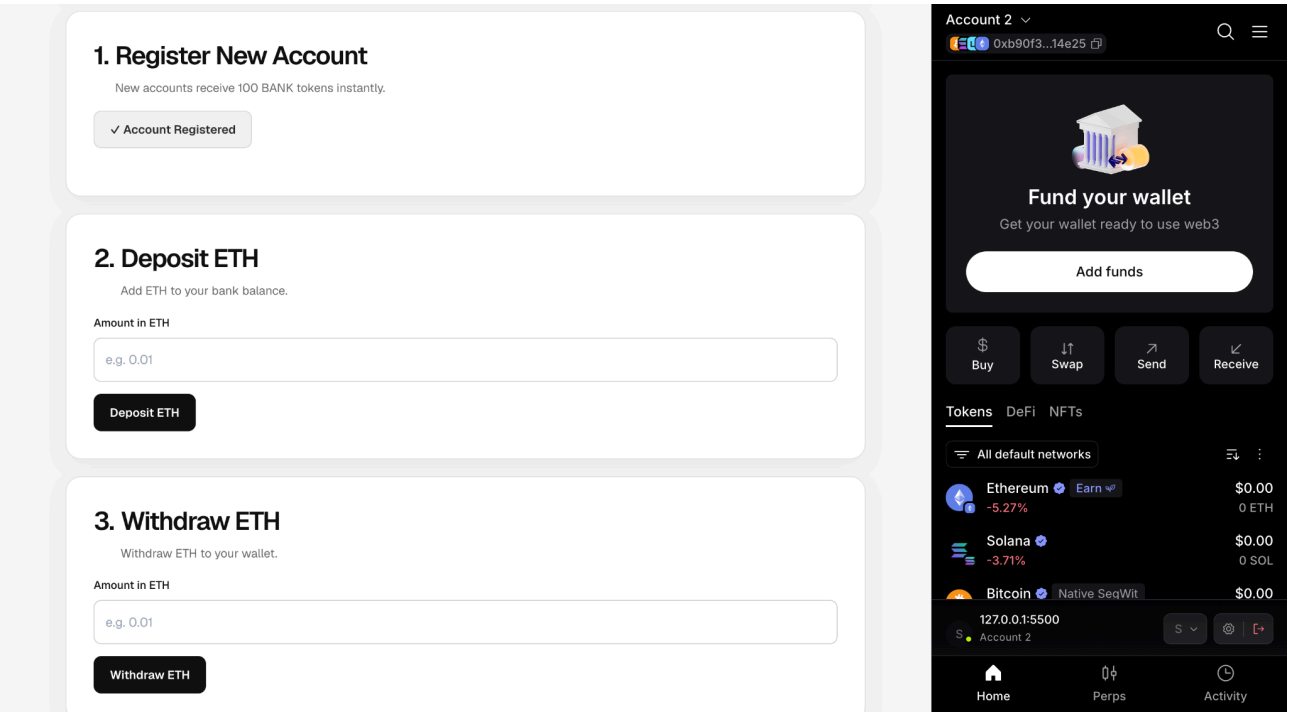


Figure 3: Smart contract requirement preventing already registered user from registering again.

TC04:

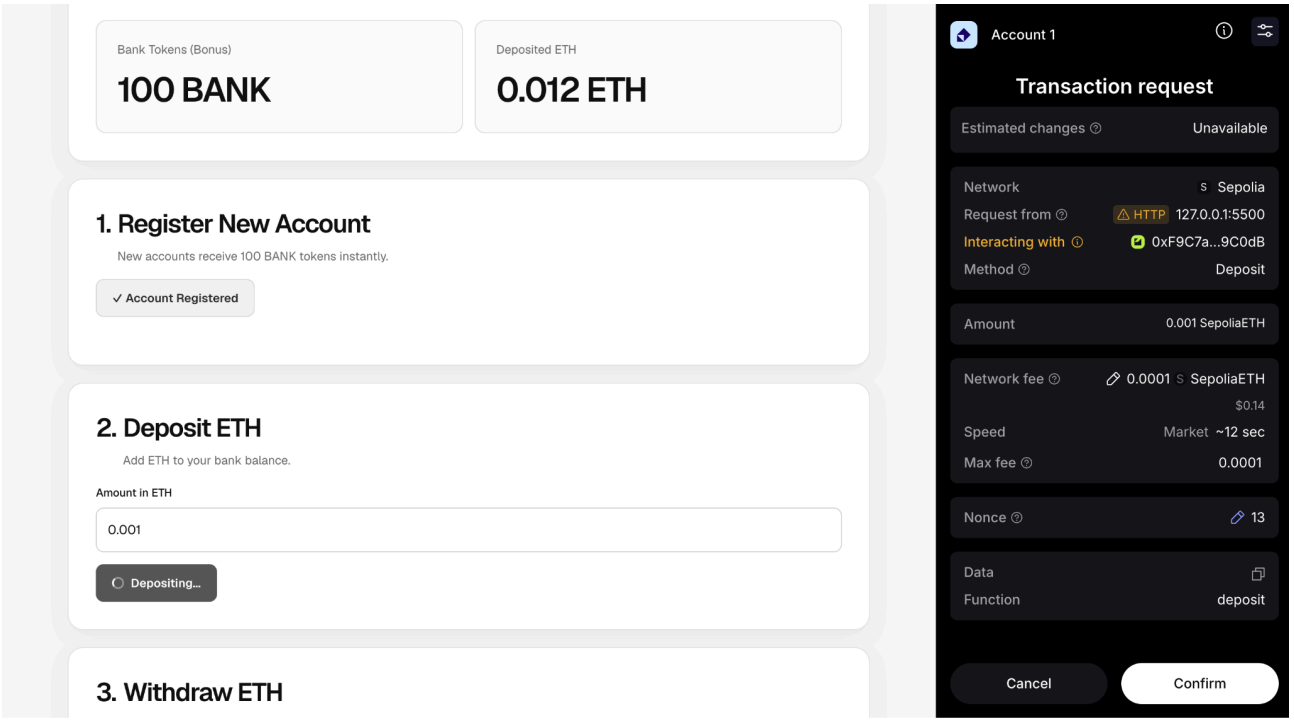


Figure 4: Depositing 0.01 Sepolia ETH into the smart contract vault via payable function.

TC05:

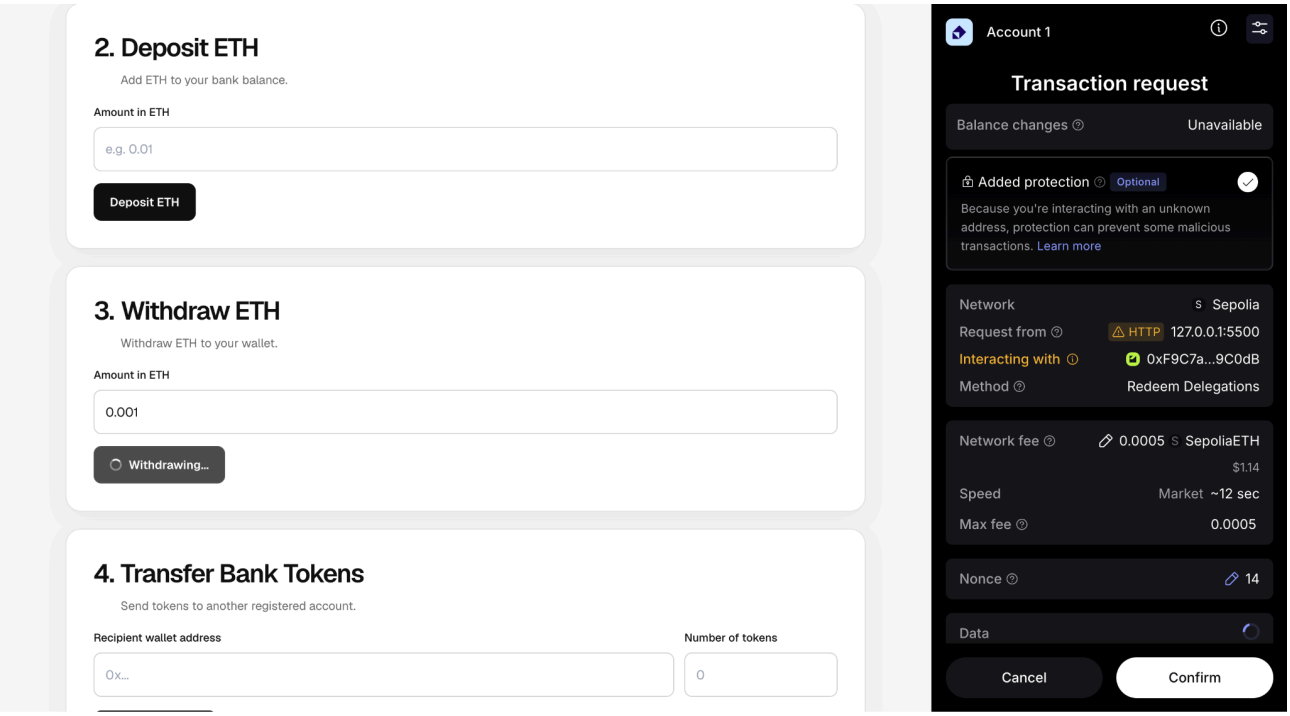


Figure 5: Withdrawing 0.01 ETH back to user's wallet with balance update.

TC06:

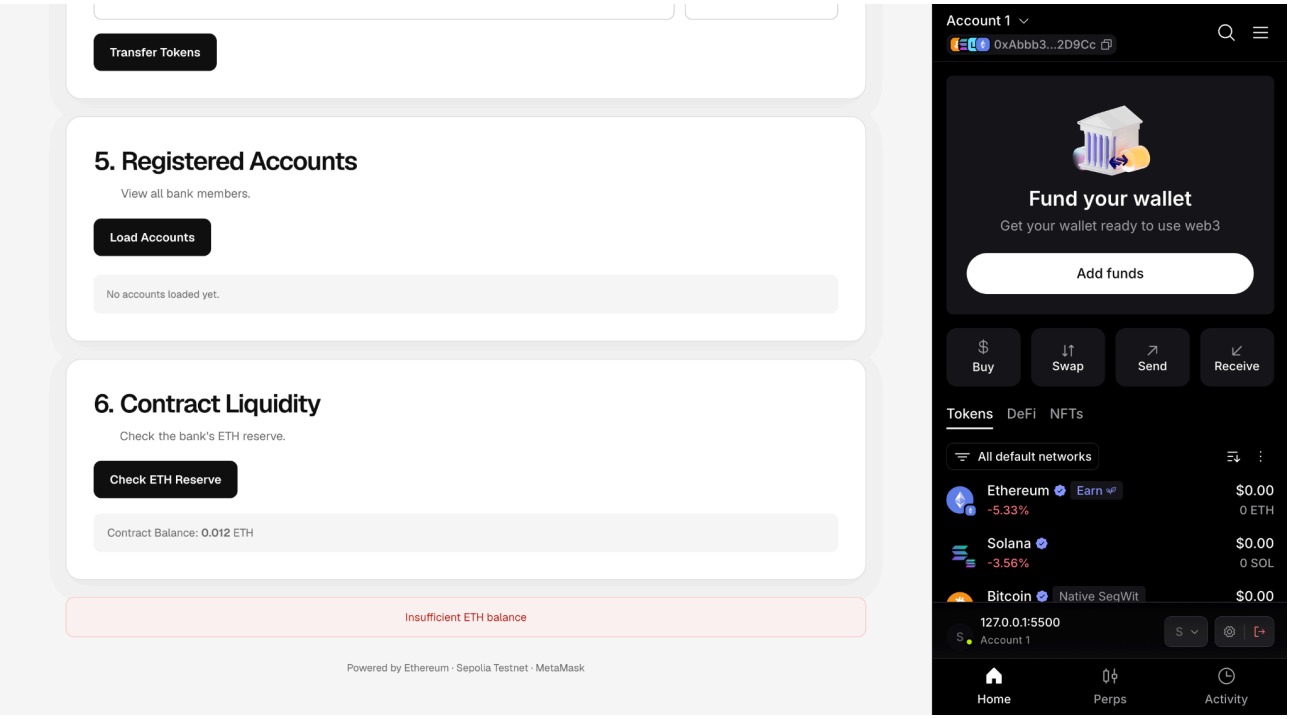


Figure 6: Smart contract revert verifying that users cannot withdraw more ETH than deposited

TC07:

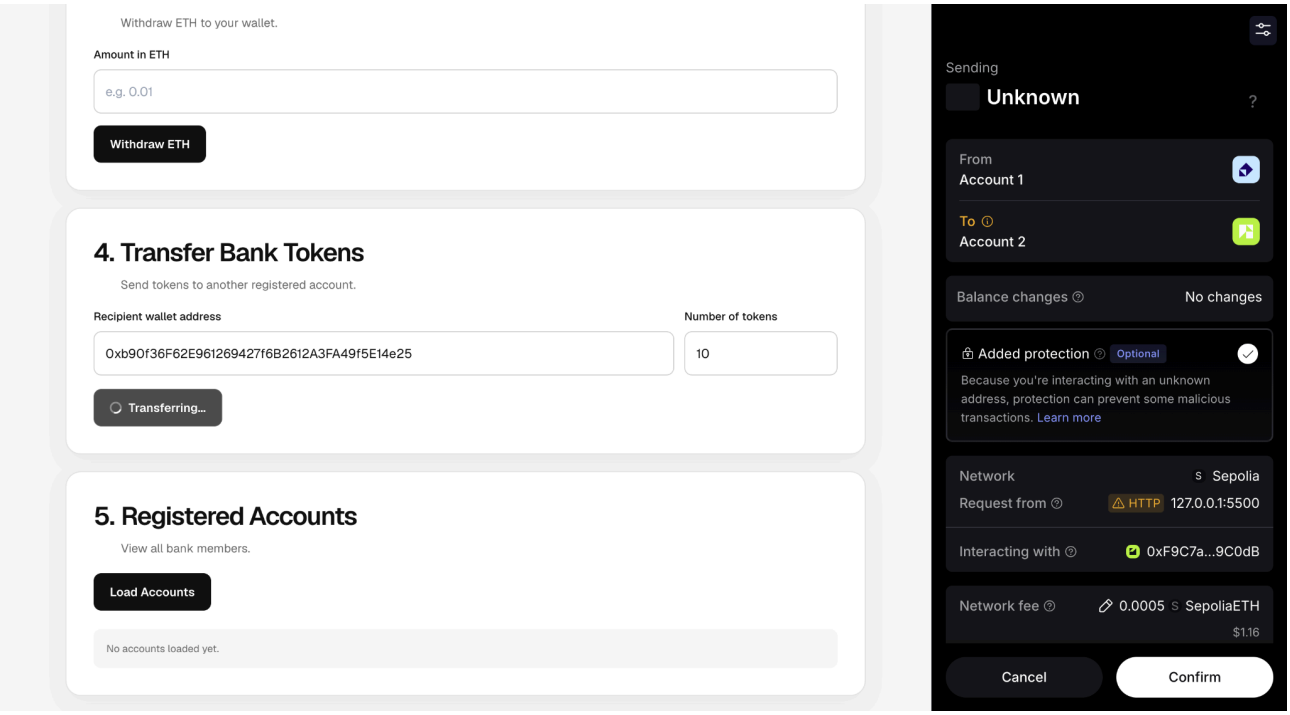


Figure 7: Successful on-chain transfer of tokens from Account 2 to Account 1.

TC08:

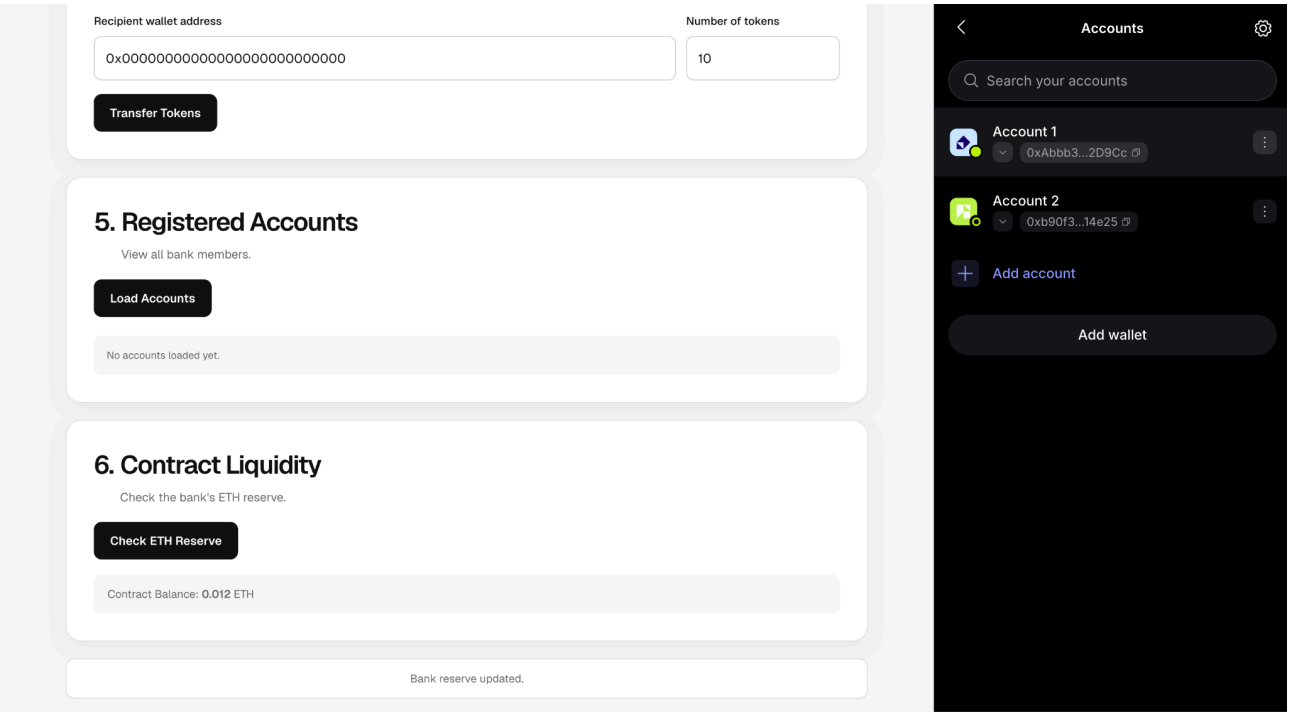


Figure 8: Security validation preventing transfer to non-whitelisted/unregistered addresses.

TC09:

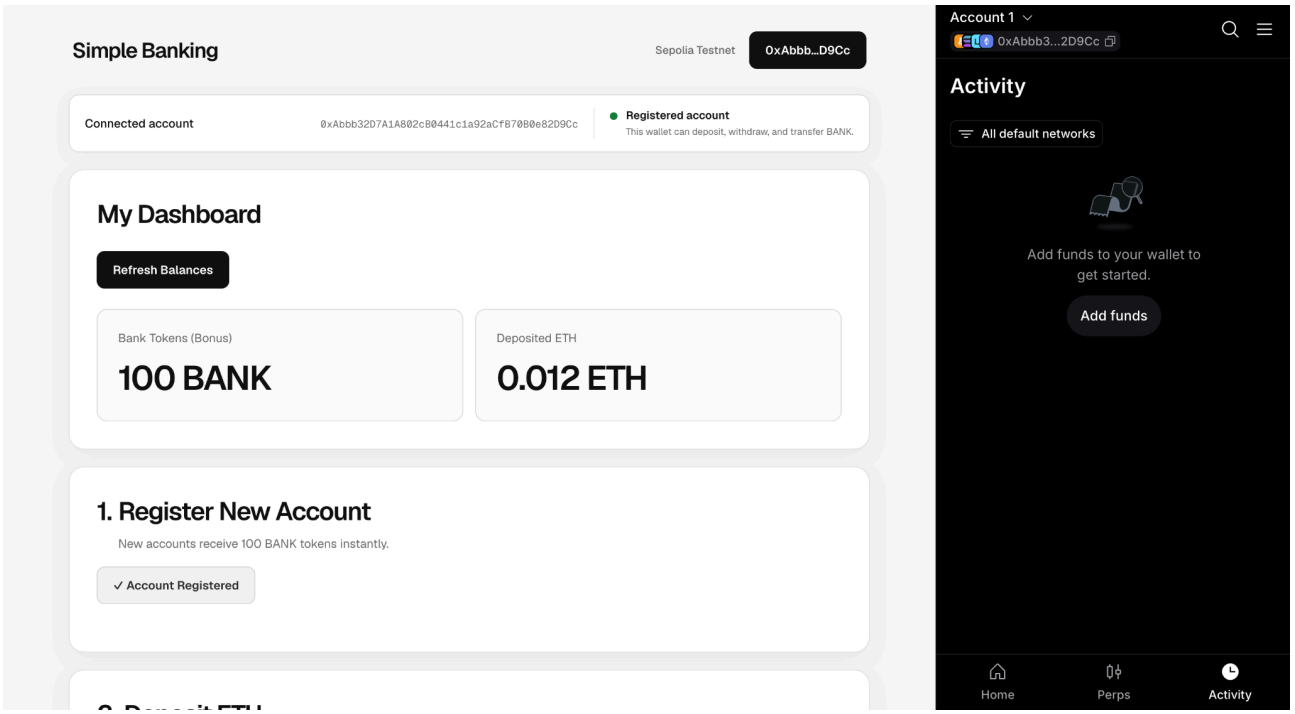


Figure 9: Balance query demonstrating Account 1 balance increased to 121 tokens after receiving transfer.

TC10:

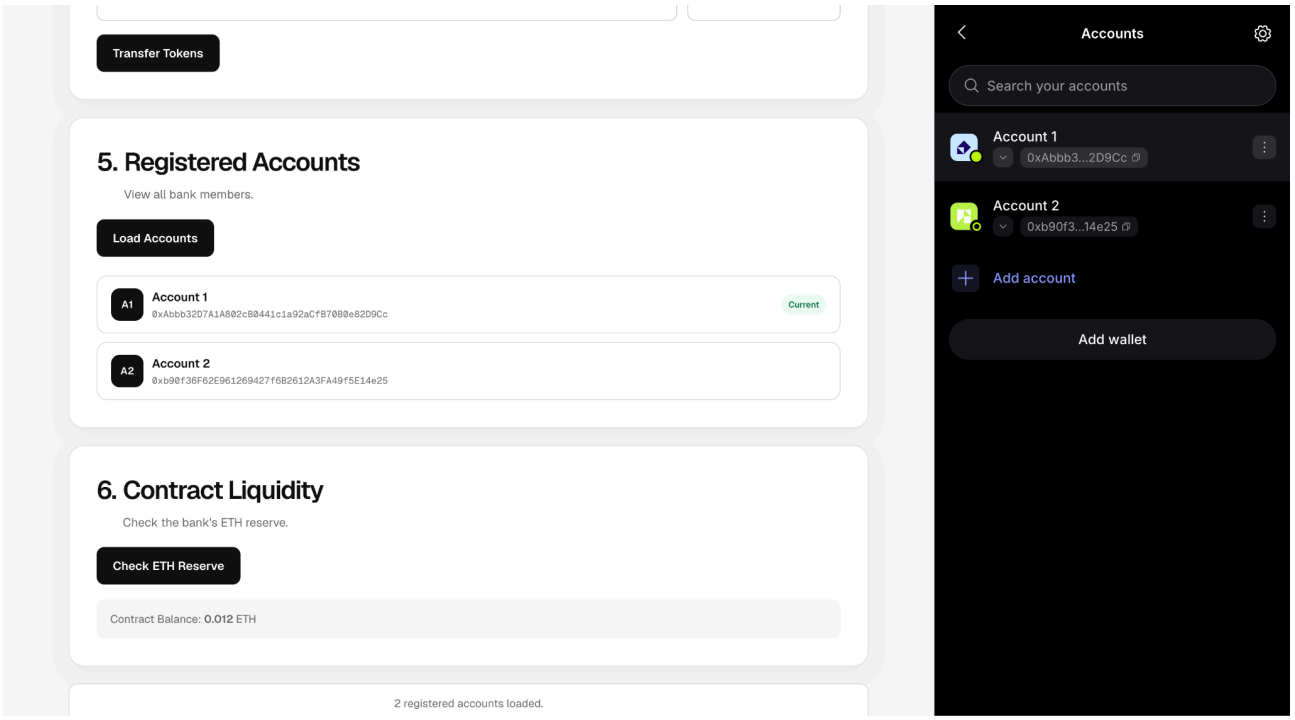


Figure 10: Dynamic list of all registered accounts and total contract liquidity reserve.